Lab 1: Requirements Table - Problem Statement #41: OpenAPI Mock Server Generator

Name: N S Raj Nanjappa | SRN: PES1UG24CS287 | Section: E | PES University, Dept. of CSE

Actors: API Developer, QA Engineer. 5 Functional Requirements (FR-001 to FR-005) and 2 Non-Functional Requirements (NFR-001, NFR-002).

ID	Type	Description	Priority	Acceptance Criteria	Rationale
FR-001	Functional	The system shall parse OpenAPI 3.0 schema definitions (YAML/JSON) and generate dynamic HTTP mock endpoints returning randomized schema-compliant JSON payloads.	High	**Pass:** Each generated endpoint responds with JSON whose fields match the types/formats declared in the spec. **Fail:** An invalid endpoint path returns 200 OK.	This is the core purpose of the tool; every other feature depends on endpoints being generated from the spec.
FR-002	Functional	The system shall validate every incoming request (path/query parameters, headers, body) against the OpenAPI schema and return a 400 response with a descriptive error body when it does not conform; unknown paths shall return 404.	High	**Pass:** A request with a wrong-typed or missing required field returns 400 naming the offending field; unknown path returns 404. **Fail:** A non-conforming request returns 2xx.	Lets QA Engineers verify that their client code sends contract-correct requests (schema validation requirement).
FR-003	Functional	The system shall allow the user to configure simulated response latency per endpoint (fixed value or min-max range, in ms) and shall delay each response accordingly.	High	**Pass:** With latency set to 300 ms, measured response times lie in 290-310 ms. **Fail:** Configured latency is ignored or applied to the wrong endpoint.	Allows developers/QA to test timeouts, loading states and retry logic without a real slow backend.
FR-004	Functional	The system shall accept OpenAPI specifications by file upload or paste, validate them for correctness, and report errors with location (line / JSON path) without creating endpoints when the spec is invalid.	Medium	**Pass:** A valid spec is accepted and listed; a spec with a broken $ref is rejected with the error location shown and no endpoints created. **Fail:** An invalid spec is partially deployed.	Prevents broken or half-built mock servers and gives fast feedback on spec mistakes.
FR-005	Functional	The system shall record every request received by a mock endpoint (timestamp, method, path, status, latency, validation result) and let the user view and filter these logs.	Medium	**Pass:** After 10 test calls, the log shows 10 entries with correct status codes and can be filtered by endpoint and status. **Fail:** Any request is missing from the log.	Gives QA Engineers evidence of what the client actually sent, which speeds up debugging and test reporting.
NFR-001	Performance	The mock server engine shall sustain 1,000 mock API requests per second with user-configured simulated latency accurate within ±10 ms.	High	**Pass:** A load test of 1,000 req/s for 5 minutes shows 95th-percentile latency error ≤ 10 ms and error rate < 0.1%. **Fail:** Latency deviation exceeds 10 ms under peak load.	Inaccurate latency or throughput collapse would make performance and timeout testing results unreliable.
NFR-002	Security	The system shall process uploaded specs safely: reject files larger than 5 MB, refuse to resolve remote (http/https) $ref URLs, never execute content from a spec, and bind the mock server to localhost unless the user explicitly enables external access.	High	**Pass:** A 10 MB spec is rejected; a spec with a remote $ref is rejected; port scan from another host finds no open port by default. **Fail:** Any of these is accepted or exposed.	Specs come from untrusted sources; unsafe parsing or exposure could lead to SSRF, denial of service or leaking internal API designs.